ENME 561 Lab 1, Task 2: Solving ODEs with Forward Euler

Name: Temuulen Ankhbayar UCID: 30188632

This script solves three different linear first-order ODEs with a hand-written Forward Euler function and verifies each one against MATLAB's built-in ode45. All three have known analytic solutions, so the true error can be measured rather than guessed at.

Open this file in MATLAB and use File > Save As to save it as a .mlx Live Script, then export to PDF (File > Export to PDF) for submission as LastName_FirstName_Q2.pdf.

```
clear; close all; clc;
```

Solve dy/dt = f(t,y) using the Forward Euler method.

[t, y] = forwardEuler(f, tspan, y0, h)

Inputs f : function handle f(t, y) returning dy/dt. Must accept y as a column vector and return a column vector of the same size, so the same code works for systems as well as scalar ODEs. tspan : [t0 tf], the integration interval. y0 : initial condition at t0 (scalar or column vector). h : step size.

Outputs t : column vector of time points. y : solution array, one row per time point (same convention as ode45).

Method y(k+1) = y(k) + h * f(t(k), y(k))

This is a first-order explicit method: the local truncation error per step is O(h^2) and the accumulated global error is O(h). Halving h should therefore halve the error.

```
function [t, y] = forwardEuler(f, tspan, y0, h)

% --- basic input checking ------------------------------------------
if ~isa(f, 'function_handle')
    error('forwardEuler:badInput', 'f must be a function handle.');
end
if numel(tspan) ~= 2 || tspan(2) <= tspan(1)
    error('forwardEuler:badSpan', 'tspan must be [t0 tf] with tf > t0.');
end
if h <= 0
    error('forwardEuler:badStep', 'h must be positive.');
end

% --- build the time grid -------------------------------------------
t = (tspan(1):h:tspan(2))';
if abs(t(end) - tspan(2)) > 1e-12          % h does not divide the interval
    t(end+1) = tspan(2);                   % add a final short step
end

y0 = y0(:);                                % force column
n  = numel(t);
y  = zeros(n, numel(y0));
```

```
y(1, :) = y0.';

% --- march forward ----------------------------------------------------
for k = 1:n-1
    dt = t(k+1) - t(k);                    % handles the ragged last step
    y(k+1, :) = y(k, :) + dt * f(t(k), y(k, :).').';
end
end
```

Section 0: the Forward Euler function

forwardEuler.m must be on the path (same folder is fine). It implements

```
y(k+1) = y(k) + h * f(t(k), y(k))
```

If you prefer everything in one file, paste the function at the very bottom of this script instead. MATLAB allows local functions at the end of a script.

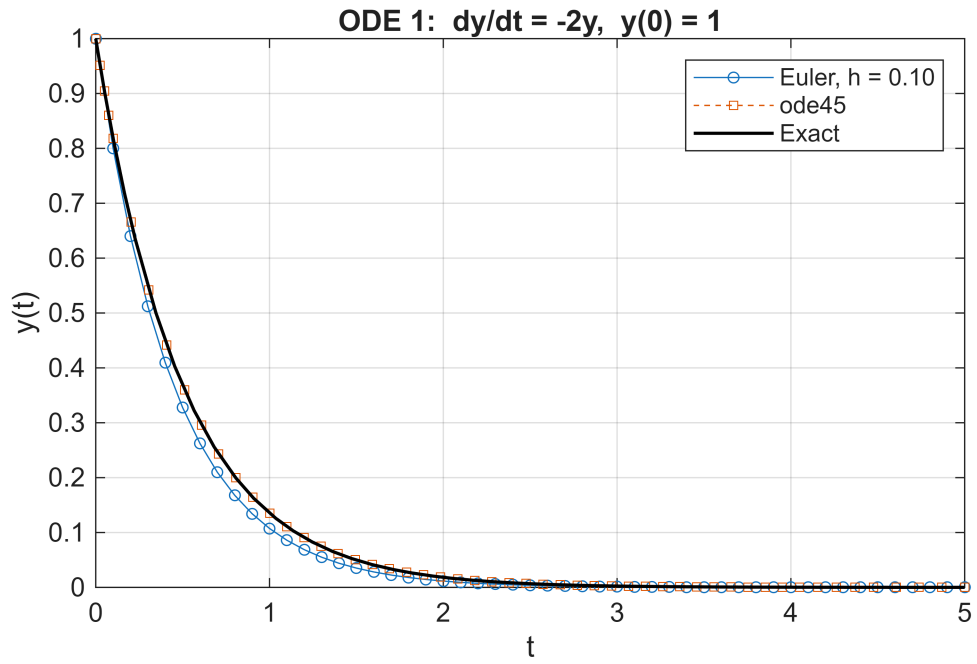
Section 1: ODE 1

```
dy/dt = -2y,   y(0) = 1,   0 <= t <= 5
exact solution: y(t) = exp(-2t)
```

```
f1     = @(t, y) -2*y;
y0_1   = 1;
tspan1 = [0 5];
exact1 = @(t) exp(-2*t);

h1 = 0.1;
[tE1, yE1] = forwardEuler(f1, tspan1, y0_1, h1);
[tO1, yO1] = ode45(f1, tspan1, y0_1);

figure;
plot(tE1, yE1, 'o-',  'MarkerSize', 4, 'DisplayName', sprintf('Euler, h = %.2f',
h1)); hold on;
plot(tO1, yO1, 's--', 'MarkerSize', 4, 'DisplayName', 'ode45');
fplot(exact1, tspan1, 'k-', 'LineWidth', 1.2, 'DisplayName', 'Exact');
grid on; xlabel('t'); ylabel('y(t)');
title('ODE 1:  dy/dt = -2y,  y(0) = 1');
legend('Location', 'northeast');
```



Section 2: ODE 2

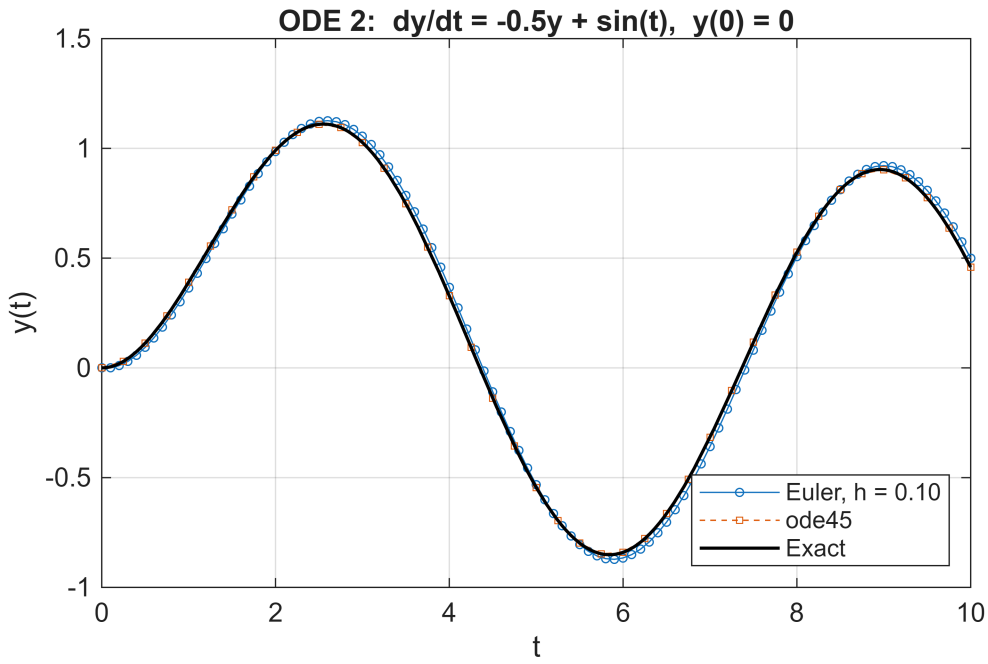
```
dy/dt = -0.5y + sin(t),   y(0) = 0,   0 <= t <= 10
exact solution: y(t) = 0.8*exp(-0.5t) + 0.4*sin(t) - 0.8*cos(t)
```

This one has a forcing term, so it checks that the solver handles an explicit t-dependence and not just y-dependence.

```
f2     = @(t, y) -0.5*y + sin(t);
y0_2   = 0;
tspan2 = [0 10];
exact2 = @(t) 0.8*exp(-0.5*t) + 0.4*sin(t) - 0.8*cos(t);

h2 = 0.1;
[tE2, yE2] = forwardEuler(f2, tspan2, y0_2, h2);
[tO2, yO2] = ode45(f2, tspan2, y0_2);

figure;
plot(tE2, yE2, 'o-',  'MarkerSize', 3, 'DisplayName', sprintf('Euler, h = %.2f',
h2)); hold on;
plot(tO2, yO2, 's--', 'MarkerSize', 3, 'DisplayName', 'ode45');
fplot(exact2, tspan2, 'k-', 'LineWidth', 1.2, 'DisplayName', 'Exact');
grid on; xlabel('t'); ylabel('y(t)');
title('ODE 2:  dy/dt = -0.5y + sin(t),  y(0) = 0');
legend('Location', 'southeast');
```



Section 3: ODE 3

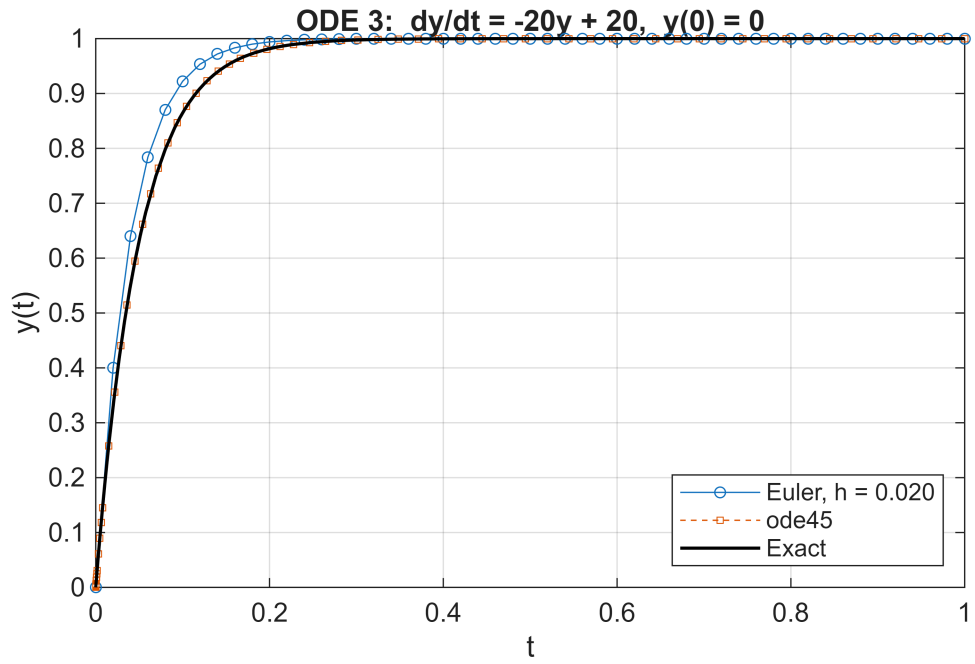
```
dy/dt = -20y + 20,   y(0) = 0,   0 <= t <= 1
exact solution: y(t) = 1 - exp(-20t)
```

The time constant is 1/20 = 0.05 s, which is short compared with the 1 s window. This is the equation that shows off Euler's step-size limit.

```
f3     = @(t, y) -20*y + 20;
y0_3   = 0;
tspan3 = [0 1];
exact3 = @(t) 1 - exp(-20*t);

h3 = 0.02;
[tE3, yE3] = forwardEuler(f3, tspan3, y0_3, h3);
[tO3, yO3] = ode45(f3, tspan3, y0_3);

figure;
plot(tE3, yE3, 'o-',  'MarkerSize', 4, 'DisplayName', sprintf('Euler, h = %.3f',
h3)); hold on;
plot(tO3, yO3, 's--', 'MarkerSize', 3, 'DisplayName', 'ode45');
fplot(exact3, tspan3, 'k-', 'LineWidth', 1.2, 'DisplayName', 'Exact');
grid on; xlabel('t'); ylabel('y(t)');
title('ODE 3:  dy/dt = -20y + 20,  y(0) = 0');
legend('Location', 'southeast');
```



Reflection question 1

State the three differential equations you chose. For each one, are the Euler and ode45 solutions the same? How do you compare them?

The three equations are:

```
(1) dy/dt = -2y,            y(0) = 0, on 0 <= t <= 5
(2) dy/dt = -0.5y + sin(t), y(0) = 0, on 0 <= t <= 10
(3) dy/dt = -20y + 20,      y(0) = 0, on 0 <= t <= 1
```

The two solutions are not identical, but they are close but not identical. I compared them by graphing.

Reflection question 2

For each equation, which solution is more accurate, and why?

ode45, for all three is more accurate. Euler is a first order method, so its error is proportional to h.

Reflection question 3

How can you increase the accuracy of the less accurate method?

The less accurate method is Forward Euler. To increase accuracy, the step size h needs to be decreased.